Simulación: el sprint de la v1.0

El caso. El banco quiere lanzar el lunes la **versión 1.0** del modelo de riesgo de crédito, que ayuda a decidir qué solicitudes se aprueban. Dos data scientists trabajan en paralelo en mejoras distintas, y todo cambio se revisa antes de llegar a producción… pero en medio del sprint aparecerá una alerta urgente. Su misión: llegar a la v1.0 sin romper producción, usando ramas, Pull Requests y GitFlow.

Roles (parejas)

Rol	Responsabilidades
DS Senior (líder técnico)	Prepara el repo y protege las ramas. Desarrolla `feature/reproducibilidad`. Revisa y fusiona el PR del Junior. Implementa el hotfix.
DS Junior	Desarrolla `feature/balanceo-clases`. Revisa el PR del Senior. Resuelve el conflicto. Revisa el hotfix. Prepara la release.

*Si un equipo tiene 3 integrantes, el tercero es **DS Junior 2** y desarrolla la misión opcional `feature/ampliar-grid` (al final de este archivo).*

Antes de empezar (ambos, una sola vez):

```
git config --global pull.rebase false
pip install -r requirements.txt
```

Ejecutar el modelo: ▶ `python run_pipeline.py` (≈ 1 minuto). La primera vez descarga los datos desde S3. Muestra el AUC en train y test.

IDE: las indicaciones usan **PyCharm**; los comandos de terminal funcionan igual en cualquier editor.

Misión 1 · Preparar el repositorio (DS Senior) · ~8 min

1. En la página de esta plantilla en GitHub: **Use this template** → **Create a new repository**.
   - Owner: la organización del curso · Nombre: `sprint-equipo-<N>` · Visibilidad: **Public**.
2. **Settings** → **Collaborators and teams**: agrega al DS Junior con permiso **Write** (si no tiene acceso ya por la organización).
3. Clona el repo y crea la rama `develop`:

```
git clone <url-del-repo>
cd sprint-equipo-<N>
git switch -c develop
```

```
git push origin develop
```

4. **Settings → General → Default branch**: cambia a `develop`. *Así los PRs apuntarán a `develop` por defecto.*
5. **Settings → Branches → Add classic branch protection rule**:
   - Branch name pattern: `main` · marca **Require a pull request before merging** y **Require approvals: 1** · **Create**.
   - Repite para `develop`.
6. Avisa al DS Junior que ya puede clonar el repo.

✅ **Listo cuando:** en la pestaña **Branches** ves `main` y `develop` con el ícono de protección.

Misión 2 · Features en paralelo (ambos) · ~10 min

El DS Junior clona el repo (quedará en `develop`):

```
git clone <url-del-repo>
cd sprint-equipo-<N>
git branch          # deberías estar en develop
```

DS Senior · `feature/reproducibilidad`

Hoy, cada ejecución da un AUC distinto porque el split y el modelo son aleatorios. En MLOps eso es un problema: no se puede auditar un modelo que no se puede reproducir.

```
git switch -c feature/reproducibilidad
```

*(En PyCharm: widget de ramas → **New Branch...**)*

1. En `src/0-split-dataset.py`, función `split_data`, fija la semilla:

```
df_data_train = df_data.sample(frac=perc_data_train, random_state=42)
```

2. En `src/3-train-evaluate-models.py`, método `train_evaluate_models`, fija la semilla del modelo:

```
rf = RandomForestClassifier(random_state=42)
```

3. Ejecuta `python run_pipeline.py` **dos veces**: el AUC debe ser idéntico en ambas.
4. Commit y push:

```
git commit -am "Fija semillas para resultados reproducibles"
git push -u origin feature/reproducibilidad
```

DS Junior · `feature/balanceo-clases`

Solo ~8% de los solicitantes incumple: las clases están desbalanceadas.

```
git switch -c feature/balanceo-clases
```

1. En `src/3-train-evaluate-models.py`, método `train_evaluate_models`, cambia la creación del modelo por:

```
rf = RandomForestClassifier(class_weight="balanced")
```

2. Ejecuta `python run_pipeline.py`.
3. Commit y push:

```
git commit -am "Agrega balanceo de clases al Random Forest"
git push -u origin feature/balanceo-clases
```

✅ **Listo cuando:** en GitHub, pestaña **Branches**, aparecen las dos ramas `feature/...`.

Misión 3 · Pull Requests y revisión de código (ambos) · ~12 min

1. **Ambos:** en GitHub, botón **Compare & pull request** de su rama. Verifica: **base:** `develop` ← compare: tu rama. Usa esta descripción:

```
## Qué cambia
## Por qué
## Evidencia (AUC antes / después)
```

2. **DS Junior:** revisa el PR del Senior → pestaña **Files changed** → deja **al menos un comentario** en una línea → **Review changes** → **Approve**.
3. **DS Senior:** fusiona su PR (**Merge pull request**).
4. **DS Senior:** abre el PR del Junior. GitHub dirá ⚠️ *This branch has conflicts that must be resolved*. ¡Era de esperarse! Ambos cambiaron la misma línea de `src/3-train-evaluate-models.py`.
5. **DS Junior** resuelve el conflicto en su computadora:

```
git switch feature/balanceo-clases
git pull origin develop
```

Aparecerá `CONFLICT (content): Merge conflict in src/3-train-evaluate-models.py`.

- En PyCharm: **Git** → **Resolve Conflicts…** → **Merge…** La línea correcta conserva **las dos** mejoras:

```
rf = RandomForestClassifier(random_state=42, class_weight="balanced")
```

- Verifica y sube la resolución:

```
python run_pipeline.py
git add src/3-train-evaluate-models.py
git commit -m "Resuelve conflicto con develop"
git push
```

6. **DS Senior:** el PR ya no tiene conflictos → revisa, aprueba y fusiona.
7. **Ambos:**

```
git switch develop
git pull
```

✅ **Listo cuando:** `develop` tiene las dos features y el pipeline corre sin errores.

Misión 4 · 🚨 Alerta: hotfix en producción (DS Senior implementa, DS Junior revisa) · ~10 min

> **Mensaje del área de Riesgos:** "El reporte del modelo en producción (`main`) muestra un **AUC de 0.27** en test, peor que lanzar una moneda. Nuestra validación independiente, con el mismo modelo, da **0.73**. Creemos que el error está en cómo se calcula la métrica. Es urgente: el comité de créditos se reúne en una hora."

El arreglo no puede esperar a la release: se hace desde `main` .

1. **DS Senior:**

```
git switch main
git pull
git switch -c hotfix/auc-probabilidades
```

2. Encuentra el error. **Pistas:**
   - El AUC se calcula en `src/4-select-best-model.py` .
   - Compáralo con `src/5-score-model.py` , que también usa `predict_proba` para generar las predicciones.
   - `predict_proba` devuelve dos columnas: `[:, 0]` = probabilidad de *no* incumplir, `[:, 1]` = probabilidad de incumplir. *(Si se atascan, la solución está al final de este archivo.)*
3. Corrige, verifica y sube:

```
python run_pipeline.py      # el AUC test debería volver a ~0.73
git commit -am "Corrige AUC usando probabilidad de clase positiva"
git push -u origin hotfix/auc-probabilidades
```

4. Abre **dos** Pull Requests desde `hotfix/auc-probabilidades` :
   - uno con **base:** `main` (para arreglar producción),
   - otro con **base:** `develop` (para que el bug no vuelva en la próxima versión).
5. **DS Junior:** revisa y aprueba ambos PRs. **DS Senior:** fusiona los dos.

✅ **Listo cuando:** `main` y `develop` tienen la corrección.

Misión 5 · Release v1.0 (DS Junior implementa, DS Senior revisa) · ~10 min

1. **DS Junior:**

```
git switch develop
git pull
git switch -c release/v1.0
```

2. En `CHANGELOG.md` , reemplaza `## [Sin publicar]` por:

```
## [1.0.0] - <fecha de hoy>

### Agregado
- Semillas fijas para resultados reproducibles.
- Balanceo de clases en el Random Forest (`class_weight`).

### Corregido
- Cálculo del AUC usando la probabilidad de la clase positiva (hotfix).
```

3. Ejecuta `python run_pipeline.py` y anota el AUC final en el CHANGELOG si quieres.
4. Commit, push y PR con **base:** `main` :

```
git commit -am "Prepara version 1.0.0"
git push -u origin release/v1.0
```

5. **DS Senior:** revisa, aprueba y fusiona el PR en `main` .
6. **DS Junior:** en GitHub, **Releases** → **Draft a new release** → tag `v1.0` sobre `main` → título "v1.0 · Modelo de riesgo de crédito" → **Publish release**.
7. Abre un último PR desde `release/v1.0` con **base:** `develop` para que el CHANGELOG también quede en `develop` . El Senior aprueba y fusiona.

✅ **Listo cuando:** el repo muestra la release **v1.0**. Abran **Insights** → **Network** para ver el dibujo de su GitFlow y peguen el link en el chat.

Misión opcional · `feature/ampliar-grid` (DS Junior 2, solo equipos de 3)

1. Desde `develop`: `git switch -c feature/ampliar-grid`
2. En `src/3-train-evaluate-models.py`, función `main()`, amplía la búsqueda:

```
model_parameters_grid = {'n_estimators': [100, 200], 'max_depth': [6, 8],
                         'min_samples_leaf': [100], 'min_impurity_decrease': [0]}
```

3. Ejecuta el pipeline (tardará un poco más), commit, push y PR hacia `develop`. Lo revisa el DS Senior.

Retos extra

- Mira la historia completa en la terminal: `git log --oneline --graph --all .`
- En PyCharm, abre la pestaña **Git (Alt + 9)** y compara el gráfico con el de **Insights** → **Network**.
- Después del hotfix, ejecuta `git blame src/4-select-best-model.py`: ¿qué commit aparece en la línea del AUC?
- Crea una plantilla de PR para el equipo: `.github/pull_request_template.md` (por una rama `feature/` y un PR, claro).

Solución del hotfix (solo si se atascan)

En `src/4-select-best-model.py`, método `_evaluate_best_model_in_dataset`, cambia:

```
auc_metric = metrics.roc_auc_score(df_data[y_col], y_pred[:,0])
```

por:

```
auc_metric = metrics.roc_auc_score(df_data[y_col], y_pred[:,1])
```

La columna 0 es la probabilidad de **no** incumplir. Usarla invierte el ranking: por eso el AUC daba $1 - 0.73 \approx 0.27$.